

QVM Racing Workbench Implementation Plan

Detailed handoff for the next implementation agent

This document defines the work required to build a separate QVM Racing Workbench from the latest saved QVM Football Workbench architecture. It protects the football Site and sets the deployment boundary before any code or infrastructure change.

Project	QVM Racing Workbench
Document	Implementation plan
Status	Implementation handoff
Protected app	QVM Football Workbench
Deployment	Separate Site only after explicit approval
Trading mode	Paper positions only

Reader action: review this document before any implementation or deployment work begins.

Contents

- Existing QVM Reference Map
- Target File Map
- Task 1 Isolate the QVM Racing Workspace
- Task 2 Define Versioned Racing Contracts and Time Semantics
- Task 3 Build the Clean Racing Persistence Layer
- Task 4 Add Provider Boundaries, Raw Snapshots, and Normalization
- Task 5 Implement As-of Features, Baselines, Model, and Calibration
- Task 6 Add Dynamic Freshness, Value, Risk, Paper Ledger, and Settlement
- Task 7 Implement the QVM Racing Worker and Control-Room Heartbeat
- Task 8 Add the Hosted QVM Racing Data and API Layer
- Task 9 Build the QVM Racing Workbench Interface
- Task 10 Implement Chronological Backtesting and Research Reporting
- Task 11 Document Operation and Optional Windows Worker Installation
- Task 12 Verify Isolation, Build, Deploy Gate, and Handoff
- Final Handoff Requirements

Global Constraints

- Treat QVM Football Workbench as protected production software. Do not modify, redeploy, migrate, reseed, or share credentials with it.
- Re-check the latest saved football commit before starting. The current reference branch is `timeout-fix`; create a separate `qvm-racing-foundation` branch or isolated repository/copy and do not add racing work to `timeout-fix`.
- Do not create a new Sites project or deploy a second Site during implementation unless Andrew explicitly authorizes that step after reviewing the working racing build.
- Never copy `.env`, `.env.windows`, `qvm.sqlite3`, `system.log`, D1 data, worker secrets, football paper bets, or football heartbeats into the racing app.
- Use the QVM name in product, environment, worker, log, API, and documentation identifiers: QVM Racing Workbench, `QVM_RACING_*`, and `qvm-racing`.
- The first release is paper-only. Do not add live order placement, exchange order submission, bookmaker credentials, or an execution button that can place a real bet.
- `WORKER_INTERVAL_SECONDS=300` is a base scheduling interval, not a quote-freshness rule.
`REQUEST_TIMEOUT_SECONDS=300` is a request/site-sync ceiling, not permission to use five-minute-old near-off quotes.
- All times crossing Python, JSON, D1, and the UI are UTC ISO-8601 values. The UI must label local display time and retain the UTC value in the audit record.
- Every model prediction must be race-grouped, chronological, calibrated, and reproducible from an immutable input snapshot. Results and final prices cannot be features for that race.
- Every new behavior gets a failing test before the minimal implementation and a focused commit after the test passes.

Review Focus

- A race with a stale quote, suspended market, missing `source_updated_at`, or a runner declared non-runner must be visibly blocked and must not create a paper position. Test in `tests/racing/test_risk.py`.
- A race-level probability vector must contain every active runner exactly once and sum to 1 within 1e-6, even when runners are withdrawn. Test in `tests/racing/test_model.py`.
- A historical prediction must not see a result, final price, post-off field, or later form snapshot. Test in `tests/racing/test_backtest.py`.
- A failed worker refresh or failed Site sync must still update heartbeat timestamps and expose `lastAttemptedSyncAt`, `lastSuccessfulSyncAt`, and the error separately. Test in `tests/racing/test_worker_service.py`.
- A replayed sync payload must be idempotent by `cycleId` and must never duplicate races, quotes, predictions, paper positions, or agent runs. Test in `tests/racing-api.test.mjs`.

Existing QVM Reference Map

The next agent must read these files in the protected QVM copy before implementing, then use them as patterns rather than importing their football assumptions:

QVM file	Reuse	Do not copy blindly
<code>app/page.tsx</code>	Dark dashboard layout, tabs, status pills, Manager + Worker control room, stale-data language	Football labels, home/draw/away tables, football-only fields
<code>db/schema.ts</code>	Drizzle/D1 naming and migration conventions	teams, fixtures, 1X2 columns, football foreign keys
<code>lib/qvm.ts</code>	Deterministic gate style, no-vig math patterns, sync idempotency, agent/audit records	Selection = HOME/DRAW/AWAY, Dixon-Coles, 15-minute fixed stale guard, football seed data
<code>lib/qvm-timeouts.ts</code>	Shared QVM timeout vocabulary and tests	Using request timeout as a data-freshness policy
<code>app/api/qvm/worker/sync/route.ts</code>	Private worker authentication and JSON error boundary	Sharing the football worker secret or sync endpoint
<code>worker_service.py</code>	Bounded worker cycle, agent ordering, site sync, local fallback	Football refresh/model functions, early heartbeat completion, implicit site success

worker_timeout.py	Long Site-sync timeout separated from provider timeout	Treating 300 seconds as acceptable near-off quote age
execution.py, calibration.py, backtesting.py	Paper ledger, calibration, chronological evaluation concepts	Football schema and match-specific assumptions
README-Windows.md, .env.windows.example	Windows installation and Task Scheduler communication	Reusing the existing QVM-Windows-Worker-v5 folder or its .env

Target File Map

All paths in the tasks below are relative to the isolated racing copy, not the protected football checkout.

File or directory	Responsibility
racing/contracts.py, lib/qvm-racing-contracts.ts, contracts/qvm-racing-sync.schema.json	One versioned JSON contract shared by the Python worker and hosted Site
racing/database.py, racing/models.py, db/schema.ts, drizzle/*	Separate local SQL and D1 racing persistence
racing/provider.py, racing/providers/demo.py, racing/providers/betfair.py, racing/normalise.py	Provider boundary, deterministic fixture provider, read-only exchange adapter, identity normalization
racing/features.py, racing/model.py, racing/calibration.py, racing/backtest.py	As-of feature construction, probability model, calibration, walk-forward evaluation
racing/market.py, racing/risk.py, racing/execution.py, racing/settlement.py	Quote freshness, value, staking, exposure, paper placement, settlement
racing/agents.py, racing/worker_service.py, racing/worker_timeout.py	QVM Worker/Manager/Auditor orchestration and reliable heartbeat/sync
lib/qvm-racing.ts, app/api/qvm/racing/**	Hosted D1 queries, deterministic gates, authenticated worker sync, API surface
components/qvm/racing/**, app/page.tsx	QVM Racing Workbench user interface
tests/racing/**, tests/racing-api.test.mjs, tests/racing-ui.test.mjs	Python and hosted behavior tests
README-QVM-RACING.md, README-Windows-Racing.md, .env.racing.example, .env.windows-racing.example	Provider setup, paper-only rules, worker operation, recovery, and deployment handoff

Task 1 Isolate the QVM Racing Workspace

Files:

- Create: docs/superpowers/specs/2026-09-19-qvm-racing-workbench.md (copy the approved specification into the isolated racing copy)
- Create: docs/qvm-racing-baseline.md
- Create: .env.racing.example
- Test: tests/racing/test_isolation_contract.py

Interfaces:

- Consumes: the latest saved QVM Football Workbench commit and its file map.
- Produces: a racing working copy with branch identity, source commit, database names, and secret names recorded before domain code changes.
- [] **Step 1: Verify the protected source before branching.**

Run in the existing football checkout:

```
git status --short --branch
git log -1 --format=%H %s'
rg -n "QVM Football Workbench|QVM_SITE_URL|QVM_WORKER_SHARED_SECRET|worker_heartbeats" README.md README-Windows.md .env.example .env.windows.example app db lib worker_service.py
```

Expected: the worktree is understood, the latest saved commit is recorded, and no uncommitted football change is overwritten.

- **[] Step 2: Create the isolated racing branch/copy.**

From the protected checkout, use the verified commit printed in Step 1:

```
RACING_BASE_COMMIT=$(git rev-parse HEAD)
git worktree add ../qvm-racing-workbench -b qvm-racing-foundation "$RACING_BASE_COMMIT"
```

The racing copy must have its own D1 binding name, local database name `qvm_racing.sqlite3`, site URL variable `QVM_RACING_SITE_URL`, and worker secret variable `QVM_RACING_WORKER_SHARED_SECRET`.

- **[] Step 3: Record the boundary.**

Create `docs/qvm-racing-baseline.md` with the verified source commit, the isolated branch name, the protected football Site URL, the new racing database/binding names, and this statement: No football database, paper position, worker secret, or worker state is copied.

Create `.env.racing.example` with only separate names and safe values:

```
QVM_RACING_DATABASE_URL=sqlite:///./qvm_racing.sqlite3
QVM_RACING_SITE_URL=
QVM_RACING_WORKER_SHARED_SECRET=
QVM_RACING_PROVIDER=demo
AUTO_PAPER_TRADES=false
QVM_RACING_KILL_SWITCH=false
```

- **[] Step 4: Add a boundary test before adding features.**

```
from pathlib import Path

def test_racing_runtime_names_are_separate():
    root = Path(__file__).parents[2]
    env_text = (root / ".env.racing.example").read_text()
    assert "QVM_RACING_SITE_URL" in env_text
    assert "QVM_RACING_WORKER_SHARED_SECRET" in env_text
    assert "QVM_SITE_URL=" not in env_text
    assert "QVM_WORKER_SHARED_SECRET=" not in env_text
```

- **[] Step 5: Run the isolation test and commit the baseline.**

Run: `python -m pytest tests/racing/test_isolation_contract.py -q`

Expected: the test passes only after the racing environment file exists. Commit with `git add docs tests .env.racing.example && git commit -m "chore: isolate QVM racing workspace"`.

Task 2 Define Versioned Racing Contracts and Time Semantics

Files:

- Create: `contracts/qvm-racing-sync.schema.json`
- Create: `racing/__init__.py`
- Create: `racing/contracts.py`
- Create: `racing/time.py`
- Create: `lib/qvm-racing-contracts.ts`
- Create: `tests/racing/test_contracts.py`
- Create: `tests/racing-contracts.test.mjs`

Interfaces:

- Consumes: `docs/superpowers/specs/2026-09-19-qvm-racing-workbench.md`.
- Produces: `RacingSyncSnapshot`, `RaceRecord`, `RunnerRecord`, `QuoteRecord`, `PredictionRecord`, `PaperPositionRecord`, `AgentRunRecord`, `HeartbeatRecord`, `parse_utc()`, and `seconds_to_off()`.
- **[] Step 1: Write contract tests for the canonical payload.**

Python test data must use one race with two active runners, one WIN quote per runner, one prediction per runner, one heartbeat, and `schemaVersion: "qvm-racing-sync.v1"`. Assert that a naive timestamp is rejected, an invalid probability is rejected, and the complete payload validates.

```

from datetime import datetime, timezone
import pytest
from racing.contracts import RacingSyncSnapshot

def make_valid_payload():
    captured = "2026-09-19T12:00:00Z"
    return {
        "schemaVersion": "qvm-racing-sync.v1",
        "cycleId": "cycle-demo-1",
        "generatedAt": captured,
        "races": [{
            "provider": "demo",
            "providerRaceId": "demo:race:1",
            "venue": "Kempton",
            "country": "GB",
            "scheduledOffAt": "2026-09-19T12:30:00Z",
            "surface": "TURF",
            "discipline": "FLAT",
            "distanceMeters": 1600,
            "going": "GOOD",
            "className": "4",
            "fieldSize": 2,
            "status": "SCHEDULED",
            "sourceUpdatedAt": captured,
        }],
        "runners": [
            {"provider": "demo", "raceProviderId": "demo:race:1", "providerRunnerId": "demo:horse:1", "horseName": "Alpha", "declarationStatus": "DECLARED"},
            {"provider": "demo", "raceProviderId": "demo:race:1", "providerRunnerId": "demo:horse:2", "horseName": "Beta", "declarationStatus": "DECLARED"},
        ],
        "quotes": [
            {"provider": "demo", "raceProviderId": "demo:race:1", "providerRunnerId": "demo:horse:1", "marketType": "WIN", "backOdds": 2.0, "capturedAt": captured, "sourceUpdatedAt": captured, "marketStatus": "OPEN"},
            {"provider": "demo", "raceProviderId": "demo:race:1", "providerRunnerId": "demo:horse:2", "marketType": "WIN", "backOdds": 2.0, "capturedAt": captured, "sourceUpdatedAt": captured, "marketStatus": "OPEN"},
        ],
        "predictions": [
            {"provider": "demo", "raceProviderId": "demo:race:1", "providerRunnerId": "demo:horse:1", "modelVersion": "racing-logit-v1", "probability": 0.5, "fairOdds": 2.0, "calculatedAt": captured, "featureSnapshotId": "snapshot-1"},
            {"provider": "demo", "raceProviderId": "demo:race:1", "providerRunnerId": "demo:horse:2", "modelVersion": "racing-logit-v1", "probability": 0.5, "fairOdds": 2.0, "calculatedAt": captured, "featureSnapshotId": "snapshot-1"},
        ],
        "paperPositions": [],
        "agentRuns": [],
        "heartbeat": {"workerName": "qvm-racing-worker", "status": "READY", "lastStartedAt": captured, "lastFinishedAt": captured, "lastAttemptedSyncAt": captured, "lastSuccessfulSyncAt": captured, "cyclesCompleted": 1},
    }

def test_sync_payload_requires_utc_and_normalized_probabilities():
    valid_payload = make_valid_payload()
    parsed = RacingSyncSnapshot.model_validate(valid_payload)
    assert parsed.schema_version == "qvm-racing-sync.v1"
    assert abs(sum(item.probability for item in parsed.predictions) - 1.0) <= 1e-6

def test_sync_payload_rejects_naive_timestamp():
    valid_payload = make_valid_payload()
    valid_payload["generatedAt"] = "2026-09-19T12:00:00"
    with pytest.raises(ValueError):
        RacingSyncSnapshot.model_validate(valid_payload)

```

- [] **Step 2: Define the JSON schema and matching types.**

Use camelCase JSON keys and these required fields:

```
export type RacingSyncSnapshot = {
  schemaVersion: "qvm-racing-sync.v1";
  cycleId: string;
  generatedAt: string;
  races: RaceRecord[];
  runners: RunnerRecord[];
  quotes: QuoteRecord[];
  predictions: PredictionRecord[];
  paperPositions: PaperPositionRecord[];
  agentRuns: AgentRunRecord[];
  heartbeat: HeartbeatRecord;
};
```

RaceRecord must contain provider, providerRaceId, venue, country, scheduledOffAt, surface, discipline, distanceMeters, going, className, fieldSize, and status. RunnerRecord must contain providerRunnerId, raceProviderId, horseName, declarationStatus, and the as-of feature fields. QuoteRecord must contain marketType, provider, capturedAt, sourceUpdatedAt, marketStatus, and at least one executable price field. PredictionRecord must contain modelVersion, probability, fairOdds, calculatedAt, and featureSnapshotId.

- [] **Step 3: Implement UTC parsing and race timing helpers.**

```
from datetime import datetime, timezone

def parse_utc(value: str) -> datetime:
    parsed = datetime.fromisoformat(value.replace("Z", "+00:00"))
    if parsed.tzinfo is None:
        raise ValueError("timestamp must include a UTC offset")
    return parsed.astimezone(timezone.utc)

def seconds_to_off(scheduled_off_at: datetime, now: datetime) -> float:
    return (parse_utc(scheduled_off_at.isoformat()) - parse_utc(now.isoformat())).total_seconds()
```

The production implementation may normalize the inputs more cleanly, but it must preserve the rejection of naive timestamps and the UTC result.

- [] **Step 4: Validate from both runtimes.**

Run: `python -m pytest tests/racing/test_contracts.py -q` and `node --test tests/racing-contracts.test.mjs`.

Expected: both runtimes accept the same valid JSON fixture and reject the same malformed timestamps/probabilities. Commit with `git add contracts racing lib tests && git commit -m "feat: define QVM racing sync contract"`.

Task 3 Build the Clean Racing Persistence Layer

Files:

- Create: `racing/database.py`
- Create: `racing/models.py`
- Create: `tests/racing/test_models.py`
- Modify: `db/schema.ts` in the isolated racing copy so it contains racing tables only
- Create: `drizzle/00xx_qvm_racing_foundation.sql`
- Modify: `drizzle/meta/_journal.json` and add the matching snapshot generated by `npm run db:generate`

Interfaces:

- Consumes: the contract fields from Task 2 and the existing QVM Drizzle/SQLAlchemy conventions.
- Produces: Race, Runner, RunnerFormSnapshot, RacingMarketQuote, RacingPrediction, RacingPaperPosition, RacingDataSnapshot, RacingModelRun, RacingAgentRun, RacingWorkerHeartbeat, and RacingDecisionReplay tables/entities.
- [] **Step 1: Write persistence tests before schema code.**

The tests must insert one race, two runners, two quotes, and two predictions, then assert that duplicate (provider, providerRaceId) races, duplicate (raceId, providerRunnerId) runners, and duplicate (raceId, runnerId, modelVersion) predictions are rejected. Assert that scheduledOffAt, capturedAt, sourceUpdatedAt, and calculatedAt are stored as UTC values.

- **[] Step 2: Define racing-only Drizzle tables.**

Prefix the physical table names with `racing_` even though the application has a separate database. At minimum define:

- `racing_races`: provider identity, venue/country, off time, race conditions, field size, status, source timestamps.
- `racing_runners`: race foreign key, provider runner identity, horse metadata, ratings, form string, declaration status, source timestamp.
- `racing_runner_form`: historical form rows with `available_at` and event date.
- `racing_market_quotes`: race/runner/provider/market identity, captured/source times, executable prices, overround, liquidity, market status, closing flag.
- `racing_predictions`: race/runner/model identity, probability, fair odds, market probability, edge, uncertainty, data quality, feature snapshot ID, calculation time.
- `racing_paper_positions`: race/runner/market, quoted/executable price, model and implied probabilities, edge, stake, approval/status/outcome, P&L, closing odds, CLV, commission, slippage, timestamps.
- `racing_data_snapshots`, `racing_source_health`, `racing_model_runs`, `racing_agent_runs`, `racing_worker_heartbeats`, and `racing_decision_replays`.

Use text timestamps for D1, notNull only where the contract requires a value, indexes on scheduled off time/status/captured time, and unique indexes for provider identities and sync idempotency.

- **[] Step 3: Mirror the same constraints in SQLAlchemy.**

Use a separate `racing.database.Base`, not the football Base, and make the local default `sqlite:///./qvm_racing.sqlite3`. The worker must never open `qvm.sqlite3` when `QVM_RACING_DATABASE_URL` is set or when the racing launcher is used.

- **[] Step 4: Generate and inspect the migration.**

Run: `npm run db:generate`.

Inspect the generated SQL and assert that it creates only `racing_*` tables and indexes. Do not run a migration against the existing football D1 binding.

- **[] Step 5: Run persistence tests and commit.**

Run: `python -m pytest tests/racing/test_models.py -q` and `npm run build`.

Expected: constraints pass, the isolated racing build can load the schema, and no football table is required by the racing app. Commit with `git add racing db drizzle tests && git commit -m "feat: add clean QVM racing persistence"`.

Task 4 Add Provider Boundaries, Raw Snapshots, and Normalization

Files:

- Create: `racing/provider.py`
- Create: `racing/providers/__init__.py`
- Create: `racing/providers/demo.py`
- Create: `racing/providers/betfair.py`
- Create: `racing/normalise.py`
- Create: `racing/source_health.py`
- Create: `tests/racing/test_normalise.py`
- Create: `tests/racing/test_providers.py`
- Create: `README-QVM-RACING.md`

Interfaces:

- Consumes: racing persistence entities and the versioned sync contract.
- Produces: `RacingProvider`, `DemoRacingProvider`, `BetfairReadOnlyProvider`, `normalize_race()`, `normalize_runner()`, `normalize_quote()`, `record_raw_snapshot()`, and `ProviderHealth`.
- **[] Step 1: Write the provider contract tests.**

```

from datetime import datetime, timedelta, timezone

def test_demo_provider_returns_stable_race_and_runner_ids():
    fixed_now = datetime(2026, 9, 19, 12, 0, tzinfo=timezone.utc)
    provider = DemoRacingProvider(clock=lambda: fixed_now)
    races = provider.list_races(fixed_now, fixed_now + timedelta(hours=2))
    runners = provider.list_runners(races[0].provider_race_id)
    assert races[0].provider_race_id == "demo:kempton:20260919:1420"
    assert [runner.provider_runner_id for runner in runners] == ["demo:horse:1", "demo:horse:2"]

def test_quote_normalization_preserves_both_source_and_capture_times():
    raw_quote = {
        "provider": "demo",
        "race_id": "demo:race:1",
        "runner_id": "demo:horse:1",
        "market_type": "WIN",
        "back_odds": 2.25,
        "captured_at": "2026-09-19T12:00:05Z",
        "source_updated_at": "2026-09-19T12:00:00Z",
        "market_status": "OPEN",
    }
    quote = normalize_quote(raw_quote)
    assert quote.source_updated_at != quote.captured_at
    assert quote.market_type == "WIN"

```

- [] **Step 2: Define the provider protocol.**

```

from datetime import datetime
from dataclasses import dataclass
from typing import Protocol

@dataclass(frozen=True)
class ProviderHealth:
    provider: str
    status: str
    checked_at: datetime
    last_success_at: datetime | None
    message: str | None

class RacingProvider(Protocol):
    name: str

    def list_races(self, start: datetime, end: datetime) -> list[dict]:
        raise NotImplementedError

    def list_runners(self, provider_race_id: str) -> list[dict]:
        raise NotImplementedError

    def list_quotes(self, provider_race_id: str) -> list[dict]:
        raise NotImplementedError

    def list_results(self, provider_race_id: str) -> list[dict]:
        raise NotImplementedError

    def health(self) -> ProviderHealth:
        raise NotImplementedError

```

The protocol is read-only and contains no `place_order`, `cancel_order`, `submit_bet`, or equivalent method. Provider implementations must return normalized dictionaries that can be validated by the Task 2 contract models.

- [] **Step 3: Implement the deterministic demo provider first.**

Give it one scheduled race, two declared runners, a withdrawn-runner fixture, changing quote timestamps, a suspended-market fixture, and a completed result. The same input clock must produce the same IDs and prices. This provider powers local development and all tests without a credential.

- [] **Step 4: Implement the read-only external adapter boundary.**

Implement `BetfairReadOnlyProvider` only for catalog/book/stream read calls when the user supplies an approved provider key. Parse runner metadata, best back/lay prices, last traded price, traded volume, projected/starting price where supplied, market status, and source update time. If a required field is absent, return a source-health failure and no actionable quote. Do not add exchange order endpoints. Do not assume The Odds API has horse-racing coverage; do not route the football `ODDS_API_URL` into this adapter.

- [] **Step 5: Normalize names, IDs, statuses, and non-runners.**

Store raw payloads immutably in `racing_data_snapshots` with content hash, provider, endpoint, captured time, available time, and status. Normalize provider IDs without replacing them, map `SCRATCHED`, `NON_RUNNER`, `WITHDRAWN`, `DECLARED`, `RUNNER`, and `FINISHED` to the internal enum, and retain the provider's original name in the raw snapshot.

- [] **Step 6: Run provider tests and commit.**

Run: `python -m pytest tests/racing/test_normalise.py tests/racing/test_providers.py -q`.

Expected: demo data is deterministic, withdrawn runners are not active, raw snapshots retain source timestamps, and the external adapter is read-only. Commit with `git add racing tests README-QVM-RACING.md` && `git commit -m "feat: add QVM racing provider boundary"`.

Task 5 Implement As-of Features, Baselines, Model, and Calibration

Files:

- Create: `racing/features.py`
- Create: `racing/model.py`
- Create: `racing/calibration.py`
- Create: `tests/racing/test_features.py`
- Create: `tests/racing/test_model.py`

Interfaces:

- Consumes: normalized races/runners/form/quotes from Tasks 3–4.
- Produces: `build_as_of_features()`, `no_vig_probabilities()`, `predict_race()`, `calibrate_predictions()`, and `ModelOutput`.
- [] **Step 1: Write leakage and probability tests.**

```

from datetime import datetime, timezone
from types import SimpleNamespace

from racing.features import build_as_of_features
from racing.model import predict_race

def test_features_exclude_form_recorded_after_scheduled_off():
    off = datetime(2026, 9, 19, 12, 30, tzinfo=timezone.utc)
    race = SimpleNamespace(id="race-1", scheduled_off_at=off)
    runner = SimpleNamespace(id="runner-1")
    form_rows = [
        SimpleNamespace(available_at=datetime(2026, 9, 18, 10, tzinfo=timezone.utc), speed_figure=82),
        SimpleNamespace(available_at=datetime(2026, 9, 19, 12, 31, tzinfo=timezone.utc), speed_figure=99),
    ]
    features = build_as_of_features(race, runner, form_rows, as_of=race.scheduled_off_at)
    assert features["latest_speed_figure"] == 82
    assert features["runs_seen"] == 1

def test_predict_race_returns_one_normalized_probability_per_active_runner():
    race = SimpleNamespace(id="race-1", scheduled_off_at=datetime(2026, 9, 19, 12, 30, tzinfo=timezone.utc))
    active_runners = [
        SimpleNamespace(id="r1", declaration_status="DECLARED"),
        SimpleNamespace(id="r2", declaration_status="DECLARED"),
    ]
    features_by_runner = {"r1": {"rating": 80}, "r2": {"rating": 70}}
    market_quotes = {"r1": 2.0, "r2": 2.5}
    output = predict_race(race, active_runners, features_by_runner, market_quotes)
    assert {item.runner_id for item in output.runners} == {"r1", "r2"}
    assert abs(sum(item.probability for item in output.runners) - 1.0) <= 1e-6
    assert all(item.fair_odds > 1.0 for item in output.runners)

```

- [] **Step 2: Implement the as-of feature builder.**

Filter every form/event row by `available_at <= race.scheduled_off_at`, not merely by event date. Exclude result position, final price, closing price, post-off going, and any field whose source timestamp is after off time. Include only features that were knowable before the race: rating, age/sex, weight, jockey/trainer identifiers, days since last run, prior form aggregates, distance/surface/going compatibility, field size, stall where relevant, and declared equipment.

- [] **Step 3: Implement the market baseline.**

`no_vig_probabilities()` must take one valid price per active runner, reject odds ≤ 1 , compute $1 / \text{odds}$, divide by the sum, and return a probability map whose values sum to 1. Use model version `market-no-vig-v1` and mark it as a benchmark, not as a hidden feature of the challenger.

- [] **Step 4: Implement ``racing-logit-v1``.**

Use a race-level softmax over utilities:

```

utility_i = beta0 + beta_rating * rating_i
            + beta_form * recent_form_i
            + beta_weight * weight_adjustment_i
            + beta_jockey * jockey_signal_i
            + beta_trainer * trainer_signal_i
            + beta_market * log(market_probability_i)
probability_i = exp(utility_i) / sum(exp(utility_j) for active runners j)

```

Store the coefficient set and feature names in `modelVersion/modelRuns`. If a feature is unavailable, use an explicit neutral value and reduce data quality; never substitute a future result. Initial coefficients may be deterministic configuration values, but the model output must identify its configuration and commit.

- [] **Step 5: Add calibration without overwriting raw probabilities.**

Fit calibration only on completed races in chronological training windows. Store raw probability, calibrated probability, calibration method, model version, sample count, and training cutoff. Use calibrated probability for paper edge only when the calibration sample threshold is met; otherwise abstain or use the raw model with a `CALIBRATION_INSUFFICIENT` flag.

- [] **Step 6: Run model tests and commit.**

Run: `python -m pytest tests/racing/test_features.py tests/racing/test_model.py -q`.

Expected: no future fields enter features, probabilities normalize after withdrawals, market baseline is reproducible, and insufficient calibration is visible. Commit with `git add racing tests && git commit -m "feat: add calibrated QVM racing probabilities"`.

Task 6 Add Dynamic Freshness, Value, Risk, Paper Ledger, and Settlement

Files:

- Create: `racing/market.py`
- Create: `racing/risk.py`
- Create: `racing/execution.py`
- Create: `racing/settlement.py`
- Create: `tests/racing/test_risk.py`
- Create: `tests/racing/test_settlement.py`

Interfaces:

- Consumes: `RaceRecord`, `QuoteRecord`, `PredictionRecord`, and the racing database entities.
- Produces: `freshness_budget_seconds()`, `quote_gate()`, `evaluate_runner()`, `calculate_stake()`, `place_paper_position()`, and `settle_paper_position()`.
- [] **Step 1: Write failure-first gate tests.**

Cover these exact cases: a quote 301 seconds old for a race 45 minutes away is blocked only when the configured 300-second budget is exceeded; a quote 61 seconds old for a race 20 minutes away is blocked; a quote 15 seconds old for a race 90 seconds away is blocked; a suspended market is blocked regardless of age; a withdrawn runner is blocked; a missing source timestamp is allowed only outside the close window with a quality penalty; a race at or after off is blocked.

- [] **Step 2: Implement the dynamic freshness policy.**

```
def freshness_budget_seconds(seconds_to_off: float) -> int:
    if seconds_to_off <= 0:
        return 0
    if seconds_to_off < 2 * 60:
        return 10
    if seconds_to_off < 10 * 60:
        return 30
    if seconds_to_off < 30 * 60:
        return 60
    return 300
```

`quote_gate()` must compare `now - max(source_updated_at, captured_at)` with the budget, reject closed/suspended/settled/void markets, reject inactive runners, and return a structured result such as `{allowed, reasonCode, ageSeconds, budgetSeconds, qualityPenalty}`. Keep the reason code in the paper audit record.

- [] **Step 3: Implement post-cost value evaluation.**

Calculate market probability from the executable price, model edge from calibrated probability, expected value after commission and slippage, and an explicit action. Do not use displayed mid-price when a back/lay executable price exists. For the first release, support only WIN; return `UNSUPPORTED_MARKET` for every other market type.

- [] **Step 4: Implement conservative staking and exposure gates.**

Start with quarter-Kelly capped by `RACING_MAX_STAKE_PCT`, `RACING_MAX_RACE_EXPOSURE_PCT`, `RACING_MAX_MEETING_EXPOSURE_PCT`, `RACING_MAX_OPEN_EXPOSURE_PCT`, and `RACING_MAX_DAILY_LOSS_PCT`. Reject zero/negative stakes, invalid odds, risk halt, kill switch, stale quotes, low data quality, and edge below `RACING_EDGE_THRESHOLD`. Keep `AUTO_PAPER_TRADES=false` by default.

- [] **Step 5: Implement paper placement and settlement.**

`place_paper_position()` must persist the quote ID, prediction ID, input snapshot ID, executable price, model probability, edge, stake, approval status, and gate reasons. `settle_paper_position()` must handle `WON`, `LOST`, `VOID`,

and `NON_RUNNER`, apply commission/slippage according to the recorded position, calculate P&L, capture closing price/CLV when available, and be idempotent when called twice.

- **[] Step 6: Run risk and settlement tests and commit.**

Run: `python -m pytest tests/racing/test_risk.py tests/racing/test_settlement.py -q`.

Expected: every failure mode blocks without inserting a paper position, accepted positions contain an immutable audit reason, and repeated settlement does not change P&L. Commit with `git add racing tests && git commit -m "feat: add QVM racing paper risk gates"`.

Task 7 Implement the QVM Racing Worker and Control-Room Heartbeat

Files:

- Create: `racing/agents.py`
- Create: `racing/worker_service.py`
- Create: `racing/worker_timeout.py`
- Create: `run_qvm_racing_worker.ps1`
- Create: `.env.windows-racing.example`
- Create: `tests/racing/test_worker_service.py`
- Create: `tests/racing/test_worker_timeout.py`

Interfaces:

- Consumes: provider, model, risk, settlement, and sync contracts from Tasks 2–6.
- Produces: `RacingWorkerService.run_once()`, `SiteClient.sync()`, `WorkerReport`, `ManagerReport`, `AuditorReport`, and a complete `HeartbeatRecord`.

- **[] Step 1: Write worker failure tests.**

Use fake provider and fake Site clients to assert that a provider exception produces `DEGRADED`, still increments the cycle, sets `lastFinishedAt`, stores `lastError`, and does not place a paper position. Use a sync exception to assert that `lastAttemptedSyncAt` changes while `lastSuccessfulSyncAt` remains unchanged. Use a successful cycle to assert both timestamps change and `cycleStatus=SUCCEEDED`.

- **[] Step 2: Implement the bounded cycle in this order.**

1. Generate a unique `cycleId` and persist `RUNNING` with `lastStartedAt`.
2. Fetch races, runners, results, and market quotes through the provider boundary.
3. Persist raw snapshots and normalized rows idempotently.
4. Build as-of features, predictions, calibration, and deterministic candidate evaluations.
5. Run Worker, Manager, and Auditor reports over deterministic facts.
6. Place a paper position only after every deterministic gate and the configured approval policy passes.
7. Build the complete `RacingSyncSnapshot` including the cycle ID and status.
8. Set `lastAttemptedSyncAt` before the outbound sync, perform the sync with the separate Site-sync timeout, then set `lastSuccessfulSyncAt` only after a 2xx response and validated response body.
9. In a `finally` block, persist `lastFinishedAt`, final status, counts, and error text so the Manager + Worker control room cannot remain on an old completion timestamp.

- **[] Step 3: Keep timeouts separate.**

Add racing-specific settings with these defaults:

```
QVM_RACING_PROVIDER_TIMEOUT_SECONDS=30
QVM_RACING_SITE_SYNC_TIMEOUT_SECONDS=300
QVM_RACING_WORKER_INTERVAL_SECONDS=300
QVM_RACING_NEAR_OFF_INTERVAL_SECONDS=15
```

QVM_RACING_SITE_SYNC_TIMEOUT_SECONDS may be derived from the existing REQUEST_TIMEOUT_SECONDS=300, but the provider request must remain bounded separately. The worker must shorten its next sleep when an actionable race is inside ten minutes, or explicitly report NEAR_OFF_REFRESH_UNAVAILABLE when the provider cannot support that cadence.

- **[] Step 4: Implement deterministic agent roles.**

Worker reports source counts, quote ages, model version, candidates, and errors. Manager ranks only candidates already accepted by deterministic gates. Auditor vetoes stale data, missing source fields, invalid probability vectors, unsupported markets, risk breaches, and any proposed live action. Agent output is explanatory JSON stored with the cycle; it is never the authority for limits.

- **[] Step 5: Add the Windows launcher without touching the old worker.**

run_qvm_racing_worker.ps1 must resolve its own directory, load .env.racing/the documented racing environment, use its own .venv, and default to qvm_racing.sqlite3. It must not reference C:\QVM-Windows-Worker-v5, copy into that folder, or use football environment names. -Once must run exactly one bounded cycle and exit with a nonzero code on a failed cycle.

- **[] Step 6: Run worker tests and commit.**

Run: python -m pytest tests/racing/test_worker_service.py tests/racing/test_worker_timeout.py -q.

Expected: the heartbeat always advances, failures are visible, sync is idempotent, and the Windows command has a separate configuration. Commit with git add racing run_qvm_racing_worker.ps1 .env.windows-racing.example tests && git commit -m "feat: add QVM racing worker heartbeat".

Task 8 Add the Hosted QVM Racing Data and API Layer

Files:

- Create: lib/qvm-racing.ts
- Create: lib/qvm-racing-timeouts.ts
- Create: app/api/qvm/racing/overview/route.ts
- Create: app/api/qvm/racing/races/route.ts
- Create: app/api/qvm/racing/races/[raceId]/route.ts
- Create: app/api/qvm/racing/refresh/route.ts
- Create: app/api/qvm/racing/scan/route.ts
- Create: app/api/qvm/racing/paper-positions/route.ts
- Create: app/api/qvm/racing/settle/route.ts
- Create: app/api/qvm/racing/performance/route.ts
- Create: app/api/qvm/racing/research/route.ts
- Create: app/api/qvm/racing/agents/route.ts
- Create: app/api/qvm/racing/worker/sync/route.ts
- Create: tests/racing-api.test.mjs

Interfaces:

- Consumes: racing Drizzle schema, RacingSyncSnapshot, and deterministic Python-worker rules ported to TypeScript for hosted paper actions.
- Produces: JSON endpoints with stable success/error shapes and a private, racing-only worker sync route.
- **[] Step 1: Write endpoint contract tests.**

Test GET /api/qvm/racing/overview with demo data, GET /api/qvm/racing/races/:raceId, POST /api/qvm/racing/scan, POST /api/qvm/racing/paper-positions, and POST /api/qvm/racing/worker/sync. Assert that an absent/wrong X-QVM-Racing-Worker-Key returns 401, malformed JSON returns 400, stale paper placement returns 409 with reasonCode: "STALE_QUOTE", and a repeated cycleId returns the original counts with idempotent: true.

- **[] Step 2: Implement `lib/qvm-racing.ts` around racing-only queries.**

Expose typed functions getRacingOverview(), listRaces(), getRaceDetail(), scanRacingCandidates(), placeRacingPaperPosition(), settleRacingPosition(), getRacingPerformance(), getRacingAgents(), and

`syncRacingWorkerSnapshot()`. Do not import `lib/qvm.ts` or write to football tables. Reuse only small generic helpers after verifying they do not carry football table names or selection enums.

- [] **Step 3: Implement explicit endpoint behavior.**
- overview: race counts, next off time, source health, quality, candidate count, risk summary, portfolio, and heartbeat.
- races: filters by date, venue, country, status, and actionable flag; never labels a stale quote as actionable.
- `races/[raceId]`: race conditions, declared runners, form provenance, latest quotes, predictions, gate reasons, and audit timestamps.
- refresh: provider refresh request with provider health and source timestamp; no paper placement.
- scan: deterministic candidate evaluation with edge, fair odds, executable odds, quote age, time-to-off, quality, and action.
- paper-positions: manual paper placement with server-side repeat of every gate; never trust client probability, odds, stake, or status.
- settle: idempotent result settlement and performance update.
- performance/research: chronological metrics, model runs, calibration, CLV, drawdown, source coverage, and decision replays.
- agents: separate last attempted/finished/successful sync fields and agent audit history.
- worker/sync: racing secret, schema validation, cycle idempotency, size cap, and transaction boundary.
- [] **Step 4: Add the racing timeout policy.**

Preserve the existing QVM timeout meanings in `lib/qvm-timeouts.ts`. Add `lib/qvm-racing-timeouts.ts` for provider, racing data load, paper action, AI explanation, and Site sync. Unit-test that paper actions and Site sync allow 300 seconds while the provider path remains 30 seconds by default.

- [] **Step 5: Run API tests and commit.**

Run: `npm run build && node --test tests/racing-api.test.mjs`.

Expected: every route builds, racing authentication is independent, stale actions fail closed, and repeated sync does not duplicate data. Commit with `git add lib app/api tests && git commit -m "feat: add hosted QVM racing API"`.

Task 9 Build the QVM Racing Workbench Interface

Files:

- Modify: `app/page.tsx` in the isolated racing copy
- Create: `components/qvm/racing/racing-dashboard.tsx`
- Create: `components/qvm/racing/race-finder.tsx`
- Create: `components/qvm/racing/race-detail.tsx`
- Create: `components/qvm/racing/paper-positions.tsx`
- Create: `components/qvm/racing/agent-control-room.tsx`
- Create: `components/qvm/racing/status-pill.tsx`
- Create: `tests/racing-ui.test.mjs`

Interfaces:

- Consumes: the hosted API response types from Task 8.
- Produces: a responsive Site titled QVM Racing Workbench with visible paper-only status and explicit data-quality controls.
- [] **Step 1: Write UI assertions before component edits.**

Assert that rendered markup contains QVM Racing Workbench, Paper only, Time to off, Quote age, Model probability, Market probability, Fair odds, Executable odds, Edge, Data quality, Manager + Worker control room, Last completed, Last successful sync, and Last attempted sync. Assert that Place live bet and Submit order do not appear.

- [] **Step 2: Port the QVM visual shell without football vocabulary.**

Keep the existing QVM dark layout, compact status pills, tab pattern, research panels, and control-room style. Rename the tabs to Dashboard, Race Finder, Paper Positions, Results, Performance, Research, Agents, and Settings.

The header and metadata must say QVM Racing Workbench; no football team, fixture, goal, home/draw/away, or Dixon–Coles copy may remain.

- **[] Step 3: Implement the race table and detail view.**

Show venue, country, scheduled off in local and UTC form, time to off, surface, distance, going, class, field size, market status, source/provider, source update time, quote age, and a stale/blocked badge. For each runner show declaration status, model probability, market no-vig probability, fair odds, executable odds, post-cost edge, uncertainty, data quality, and action.

- **[] Step 4: Implement paper ledger and audit states.**

Show approval status, gate reasons, stake, price, commission/slippage, outcome, P&L, closing price, CLV, and timestamps. Make blocked states inspectable rather than hiding them. Disable paper action when the API says stale, suspended, unsupported, low quality, risk halted, or non-runner.

- **[] Step 5: Implement the control room with correct timestamp semantics.**

Render cycle status, cycle ID, worker name, current model version, source refresh counts, error text, lastStartedAt, lastFinishedAt, lastAttemptedSyncAt, and lastSuccessfulSyncAt. Label missing successful sync distinctly from an old successful sync. This directly prevents the current QVM-style “Last completed 13 Sept 16:03” ambiguity.

- **[] Step 6: Run UI tests and commit.**

Run: `npm run build && node --test tests/racing-ui.test.mjs`.

Expected: the interface is responsive, the paper-only boundary is visible, and no football or live-order control leaks into the racing copy. Commit with `git add app components tests && git commit -m "feat: add QVM racing workbench UI"`.

Task 10 Implement Chronological Backtesting and Research Reporting

Files:

- Create: `racing/backtest.py`
- Create: `racing/metrics.py`
- Create: `tests/racing/test_backtest.py`
- Create: `tests/racing/test_metrics.py`
- Modify: `app/api/qvm/racing/research/route.ts`
- Create: `docs/qvm-racing-model-card.md`

Interfaces:

- Consumes: historical normalized rows, as-of features, model versions, quotes, results, and paper settlement rules.
- Produces: `run_walk_forward_backtest()`, `calculate_calibration_metrics()`, `calculate_paper_performance()`, `ModelRunReport`, and the QVM Racing model card.
- **[] Step 1: Write the chronological split test.**

Build three races on consecutive dates where the third race contains a form row and closing price recorded after its off time. Assert that the first evaluation window contains only earlier races, that all runners of one race share a split, and that the third race's future fields are absent from its feature snapshot.

- **[] Step 2: Implement race-grouped walk-forward evaluation.**

Sort by scheduled off, fit only on races before each evaluation window, predict the entire next race, settle only after the result, and roll the window forward. Persist train cutoff, evaluation window, model version, feature manifest, source snapshot IDs, and git commit. Do not randomize runner rows.

- **[] Step 3: Implement the required metrics.**

Calculate log loss, Brier score, expected calibration error, top-ranked hit rate, abstention rate, quote/source coverage, net paper ROI after commission/slippage, turnover, maximum drawdown, average edge, and closing-line value. Compare `racing-logit-v1` with `market-no-vig-v1` and favorite baseline. Keep model accuracy separate from wagering performance.

- **[] Step 4: Write the model card.**

Document training data cutoff, fields, known leakage risks, calibration method, market assumptions, freshness gates, costs, abstention rules, sample size, out-of-sample metrics, and the statement that results do not establish future profitability or authorize real-money execution.

- [] **Step 5: Run research tests and commit.**

Run: `python -m pytest tests/racing/test_backtest.py tests/racing/test_metrics.py -q`.

Expected: future data is rejected, baselines are reported, and metrics are reproducible from stored snapshots. Commit with `git add racing app/api/qvm/racing/research/route.ts docs tests && git commit -m "feat: add QVM racing walk-forward research"`.

Task 11 Document Operation and Optional Windows Worker Installation

Files:

- Create: README-QVM-RACING.md or update the racing copy's existing README with the final instructions
- Create: README-Windows-Racing.md
- Create: .env.racing.example
- Modify: .env.windows-racing.example
- Create: install_windows_racing.ps1
- Create: tests/racing/test_documented_configuration.py

Interfaces:

- Consumes: the final environment names, worker command, provider adapter, hosted endpoints, and timeout/freshness policy.
- Produces: a reproducible paper-only setup that does not overwrite the existing QVM-Windows-Worker-v5 installation.
- [] **Step 1: Add the separate environment contract.**

Include these names with safe defaults and no credentials:

```
QVM_RACING_DATABASE_URL=sqllite:///./qvm_racing.sqlite3
QVM_RACING_SITE_URL=
QVM_RACING_WORKER_SHARED_SECRET=
QVM_RACING_PROVIDER=demo
QVM_RACING_PROVIDER_TIMEOUT_SECONDS=30
QVM_RACING_SITE_SYNC_TIMEOUT_SECONDS=300
QVM_RACING_WORKER_INTERVAL_SECONDS=300
QVM_RACING_NEAR_OFF_INTERVAL_SECONDS=15
RACING_EDGE_THRESHOLD=0.03
RACING_MAX_STAKE_PCT=0.02
RACING_MAX_RACE_EXPOSURE_PCT=0.05
RACING_MAX_MEETING_EXPOSURE_PCT=0.10
RACING_MAX_OPEN_EXPOSURE_PCT=0.25
RACING_MAX_DAILY_LOSS_PCT=0.03
AUTO_PAPER_TRADES=false
QVM_RACING_KILL_SWITCH=false
```

Document that REQUEST_TIMEOUT_SECONDS=300 can be used as a site-sync ceiling but must not override the dynamic freshness table.

- [] **Step 2: Document hosted-only and worker-assisted modes.**

Hosted-only mode uses the demo or approved provider from the Site runtime and is suitable for pre-race batch refresh. Worker-assisted mode uses outbound HTTPS only, stores local state in `qvm_racing.sqlite3`, and synchronizes through `/api/qvm/racing/worker/sync`. Explain that near-off or stream data may require a continuously running worker/provider connection; a five-minute worker interval cannot guarantee ten-second freshness.

- [] **Step 3: Document the Windows command without touching the old worker.**

The documented command must be:

```
Set-Location C:\QVM-Racing-Workbench
.\run_qvm_racing_worker.ps1 -Once
```

Include separate installation, update, log, and recovery instructions. State explicitly that the racing worker is not installed into C:\QVM-Windows-Worker-v5 and must not copy over the football worker's .env, database, or logs.

- **[] Step 4: Test the documented configuration and commit.**

Run: `python -m pytest tests/racing/test_documented_configuration.py -q`.

Expected: all documented variable names exist, no football secret names are required, and the command points to a separate folder. Commit with `git add README-QVM-RACING.md README-Windows-`

`Racing.md .env.racing.example .env.windows-racing.example install_windows_racing.ps1 tests` && `git commit -m "docs: document QVM racing operation"`.

Task 12 Verify Isolation, Build, Deploy Gate, and Handoff

Files:

- Create: `tests/qvm-racing-isolation.test.mjs`
- Create: `docs/qvm-racing-release-checklist.md`
- Modify: `README-QVM-RACING.md`

Interfaces:

- Consumes: all implementation tasks and the protected football Site URL.
- Produces: a release decision, test evidence, commit list, and a deployment-ready artifact without automatically creating a Site.
- **[] Step 1: Add a static isolation test.**

Assert that the racing API imports `db/schema.ts` racing tables only, racing routes do not import `lib/qvm.ts`, no source file contains the football D1 binding name, and no source file exposes `placeOrder`, `submitBet`, `cancelOrder`, or equivalent live execution methods. The test may allow the words `paper` and `paper-only`.

- **[] Step 2: Run the complete local verification suite.**

Run:

```
python -m pytest tests/racing -q
npm test
npm run lint
npm run build
node --test tests/racing-contracts.test.mjs tests/racing-api.test.mjs tests/racing-ui.test.mjs tests/qvm-racing-isolation.test.mjs
```

Expected: every racing test passes, the copied football baseline still builds, no secret is printed, and no test creates a live provider order.

- **[] Step 3: Verify the protected football application separately.**

In the original football checkout, run its existing build/test commands without changing files. Record the branch, commit, build result, and existing Site URL in `docs/qvm-racing-release-checklist.md`. If the football build differs after the racing copy was made, stop and report the difference; do not “fix” the football Site as part of racing work.

- **[] Step 4: Perform a demo smoke test.**

Start the isolated racing app with the demo provider, run one worker cycle, load the dashboard, inspect a race detail, verify a candidate is blocked when its quote is stale, create one manual paper position with a fresh quote, settle it, and confirm the performance and decision-replay records. Capture the cycle ID and the four heartbeat timestamps.

- **[] Step 5: Stop at the deployment gate.**

Do not create a Site, D1 database, Site secret, Windows ZIP, or public URL in this task unless Andrew explicitly approves the separate QVM Racing Workbench deployment. If approval is given later, use a separate private Sites project, separate D1 binding, separate `QVM_RACING_WORKER_SHARED_SECRET`, demo seed first, then provider credentials, and a smoke test before sharing the URL. Never point the racing worker at the football URL or `/api/qvm/worker/sync`.

- **[] Step 6: Create the final handoff commit.**

Record each commit, test command/result, provider coverage limitations, model version, database/binding names, worker command, and remaining risks in docs/qvm-racing-release-checklist.md. Commit with `git add tests docs README-QVM-RACING.md && git commit -m "chore: verify QVM racing handoff"`.

Final Handoff Requirements

The next agent's final response must include:

1. The protected QVM Football branch/commit and confirmation that its Site/data were not changed.
2. The isolated racing branch/commit and a short list of changed files.
3. Test commands and actual pass/fail results.
4. The provider used, source freshness behavior, and known coverage limitations.
5. The active model version, calibration status, and walk-forward comparison against the market baseline.
6. The exact paper-only risk settings and any blocked-candidate examples.
7. Worker heartbeat values: last start, last finish, last attempted sync, last successful sync, cycle ID, and last error.
8. A statement that no live order path or real-money credential was added.
9. Whether deployment was intentionally held for approval or completed only after explicit authorization.